

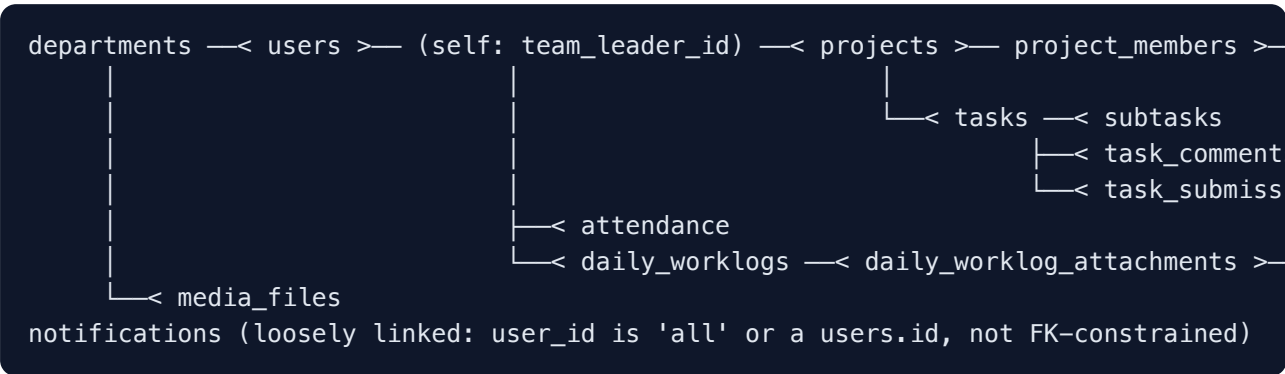
Database Architecture

OmniWork's backend is a single Supabase (hosted Postgres) project. This document describes the schema, the auth/identity model, row-level security, database triggers, and the small server-side layer that needs elevated (service-role) privileges. It reflects the live project state (verified directly against the database), not just the bootstrap SQL files.

1. Tech stack

- **Postgres** (via Supabase) — all application data.
- **Supabase Auth** (`auth.users`) — login credentials (email/password), completely separate from the `public.users` profile table (see §3).
- **Row Level Security (RLS)** on every `public` table — the database itself enforces who can read/write what; the app's anon key is safe to ship to the browser because of this.
- **Next.js Route Handlers** (`app/api/admin/**`) — the only part of the app that uses the Supabase **service-role** secret key, for operations RLS can't express (creating/deleting login credentials for other people).

2. Entity overview



All primary keys are human-readable `text` (e.g. `'dept-webdev'`, `'task-3'`), not `uuid` — a deliberate choice (see `supabase/schema.sql` header comment) so seed data and the app's existing `id=new-$(Date.now())` generation pattern didn't need to change when the schema was introduced.

3. Tables

Table	Purpose	Key columns
<code>departments</code>	Business units (SBUs)	<code>id</code> , <code>name</code> , <code>code</code> , <code>status</code>
<code>users</code>	App profile — not the login itself	<code>id</code> , <code>auth_id</code> (→ <code>auth.users.id</code> , nullable, unique), <code>email</code> (unique), <code>role</code> (nullable), <code>department_id</code> ,

Table	Purpose	Key columns
		team_leader_id (self-FK), status , performance_score
projects	Departmental projects	department_id , leader_id , assignee_id , status , progress
project_members	Many-to-many project ↔ user	composite PK (project_id, user_id)
tasks	Work items under a project	project_id , department_id , assigned_to , status , priority , position (manual ordering)
subtasks	Children of a task	task_id , owner_id , assigned_to
task_comments	Free-text comments on a task	task_id
task_submissions	Work handed in against a task	task_id , user_id , status (Pending/Approved/Changes Requested)
task_submission_attachments	Join table: submission ↔ media	composite PK
media_files	Uploaded documents/images/videos	project_id , department_id (either/both nullable)
attendance	Daily check-in/out	user_id , date , check_in_time , check_out_time , working_hours
notifications	In-app notifications	user_id (not FK-constrained — 'all' is a valid sentinel value)
daily_worklogs	End-of-day work logs	user_id
daily_worklog_attachments	Join table: worklog ↔ media	composite PK

Enums used: user_role (Admin / Team Leader / Team Member), active_status , project_status , task_status , task_priority , task_category , attendance_status , submission_status , media_type , notification_type .

Orphaned enums: employee_status and leave_status exist in the database but aren't used by any column or referenced anywhere in supabase/*.sql or the app — leftovers from an earlier template, not part of this app's design. Harmless, but flagged here in case they're ever a source of confusion.

4. Identity model: `auth.users` VS `public.users`

These are two separate tables that only meet at one column:

```
auth.users (Supabase-managed: email, password hash, confirmation state)
  |
  | users.auth_id (nullable uuid, unique, references auth.users(id) on delete s
  ▼
public.users (app-managed: name, role, department, title, performance_score, ...)
```

- A `users` row can exist **before** any login exists for it (e.g. seed data for people who never got an account) — `auth_id` is simply `null`.
- A `users` row's `role` can also be `null` — see §6, this is the "signed up, not yet categorized by an Admin" state.
- `id` (the `users` primary key) is a separate, app-generated string — never the auth UUID directly, though the signup trigger derives it from the auth UUID (`'user-' || <auth-uuid-without-dashes>`) for accounts created that way.

Two helper functions, used throughout RLS policies, resolve "who is making this request":

```
current_app_role() returns user_role    -- select role from users where auth_id = aut
current_app_user_id() returns text      -- select id    from users where auth_id = aut
```

Both are `security definer`, `stable`, `search_path = public`.

5. Row Level Security

RLS is enabled on every `public` table. The overall philosophy (stated directly in `supabase/schema.sql`): **reads are broad, writes are gated** — any authenticated user can `SELECT` from any table, and the app computes fine-grained per-role visibility client-side (`visibleTasks`, `visibleUsers`, etc. in `app/page.tsx`). Writes are where RLS actually restricts things:

Table	Insert	Update	Delete
<code>departments</code>	Admin only	Admin only	Admin only
<code>users</code>	Admin or Team Leader	Admin, or the row owner (<code>auth_id = auth.uid()</code>)	Admin only
<code>projects</code>	Admin or Team Leader	Admin or Team Leader	Admin or Team Leader
<code>project_members</code>	Admin or Team Leader (all commands)	—	—

Table	Insert	Update	Delete
attendance	Self, or Admin/Team Leader	Self, or Admin/Team Leader	—
everything else (tasks, subtasks, comments, submissions, media, worklogs, notifications)	any authenticated user	any authenticated user	any authenticated user (where delete exists)

The one policy worth calling out: **"admin or self update users"** allows *any* signed-in user to update *their own* `users` row with no column-level restriction in the policy itself. That's intentionally broad at the RLS layer and is narrowed by a trigger instead (§6) — otherwise a Team Member could `PATCH` their own row's `role` to `'Admin'` directly against PostgREST.

6. Self-signup, categorization, and the privilege-escalation guard

Added on top of the original schema to support in-app signup (`supabase/signup.sql`):

1. **`users.role` is nullable.** `NULL` means "has an account, not yet assigned a role by an Admin" — the app shows an "Awaiting Admin Approval" screen for these users (`app/page.tsx`) instead of the dashboard, and Admins see them in a "Pending Approvals" panel (`components/UsersView.tsx`) to assign role/department/team leader.
2. **`handle_new_signup()` trigger function**, meant to fire `after insert` on `auth.users` , auto-creates the matching `public.users` row (`role = null` , `department_id = null`) so nobody has to manually copy a UUID between the Dashboard and SQL Editor (`supabase/link-accounts.sql` describes the old manual process). If the new signup's email already matches an existing `users` row, it deliberately fails closed with `"An account with this email already exists. Contact your administrator."` rather than silently taking over that identity — this prevents someone from signing up with a known coworker's email and inheriting their existing role.
3. **`prevent_self_privilege_escalation()` trigger function**, meant to fire `before update` on `users` , blocks a *non-admin* user from changing their *own* `role` , `department_id` , `team_leader_id` , or `status` — this is what keeps the broad "self update" RLS policy above from being a privilege-escalation hole.

⚠ **Current live status: neither trigger is actually attached right now.** Both functions exist in the database, but `on_auth_user_created` (on `auth.users`) and `enforce_self_update_columns` (on `users`) were dropped during manual account-linking work in the Supabase Dashboard (creating `admin@enterprise.com` / `admin@123.com` required temporarily disabling the signup trigger, since those emails already existed as seeded rows) and never re-attached. Practical effect today: **new self-signups do not get an auto-created profile row**, and a non-admin could in principle self-update their own role via a direct REST call. To

restore both protections, re-run the two `create trigger` statements at the bottom of `supabase/signup.sql`.

7. Privileged server-side layer (service role)

Two operations can't be expressed through RLS at all, because they act on `auth.users` itself (creating a login with a chosen password, deleting a login) rather than on application data:

- `app/api/admin/create-user/route.ts` — Admin adds an employee *with* a password: creates the `auth.users` row via `auth.admin.createUser()`, then inserts the linked `public.users` row, rolling back the auth account if the profile insert fails.
- `app/api/admin/delete-user/route.ts` — Admin deletes an employee: removes the `auth.users` login via `auth.admin.deleteUser()`, then deletes the `public.users` row.

Both routes:

- Run only server-side (Next.js Route Handlers), using `lib/supabase/adminClient.ts`, which holds the `SUPABASE_SERVICE_ROLE_KEY` secret (server-only env var, never `NEXT_PUBLIC_`, guarded by the `server-only` package so an accidental client-component import fails the build instead of shipping the secret to the browser).
- Re-verify the caller is actually an Admin server-side (`lib/supabase/adminAuth.ts` looks up the caller's own `users.role` from their bearer token) — they never trust a client-asserted role.
- Are called from the browser via `lib/supabase/adminApi.ts`, a thin `fetch()` wrapper that attaches the caller's current session token.

Everything else in the app talks to Postgres directly from the browser via the anon key (`lib/supabaseClient.ts`) and relies entirely on RLS — no other privileged path exists.

8. App-level data access layer

The app never issues raw SQL from components. Two files own the entire Supabase surface:

- `lib/supabase/mappers.ts` — `fetchAllData()` (bulk read, joins in the `select()` strings) and `fetchCurrentProfile()` (the signed-in user's own row) convert `snake_case` DB rows into the `camelCase` shapes in `lib/types.ts`. `fetchAllData` explicitly filters `role is not null` so pending signups never leak into the normal directory/dashboards; a separate `fetchPendingUsers()` query (`role is null`) feeds the Admin-only pending-approvals panel.
- `lib/supabase/mutations.ts` — one `dbInsertX` / `dbUpdateX` / `dbDeleteX` function per table, each taking the app's `camelCase` shape and converting it back to a `snake_case` row. Callers in `app/page.tsx` invoke these, then patch local React state directly — there's no refetch after a write.

9. `supabase/*.sql` file map

Run in this order for a from-scratch setup:

File	Purpose
<code>schema.sql</code>	Full bootstrap: enums, tables, RLS policies, <code>current_app_role</code> / <code>current_app_user_id</code> helpers.
<code>seed.sql</code>	Demo data (departments, users, projects, tasks, etc.) matching the app's original hardcoded mock data.
<code>link-accounts.sql</code>	Manual process: after creating the 3 demo logins in the Dashboard, paste their UUIDs in to set <code>users.auth_id</code> . Superseded for <i>new</i> signups by <code>signup.sql</code> 's trigger, but still the reference for manually linking any pre-seeded row.
<code>signup.sql</code>	Adds nullable <code>role</code> , the auto-provisioning trigger, and the self-escalation guard trigger (see §6 for current live status).
<code>fix-auth-trigger.sql</code>	Diagnostic/cleanup for a <i>different</i> , unrelated leftover trigger from a generic Supabase starter template (<code>on_auth_user_created</code> → a nonexistent <code>public.profiles</code> table) that pre-dated this app's own signup trigger of the same name. Safe to re-run; it's a diagnostic, not part of normal setup.
<code>reset.sql</code>	Wipes and re-seeds data for a clean demo state.